

1. O que é Trinnity Viseron System?

Trinnity Viseron System (TVS) é um Sistema Operativo Multi-Agente de Superinteligência Artificial — uma plataforma autônoma que orquestra milhares de mentes artificiais para resolver problemas complexos, gerar código, criar aplicações, gerenciar tokens, automatizar fluxos de trabalho e muito mais.

Liderado pelo Comandante Supremo Pedro Costa e pela Reina Arquitecta Trinnity Hurtado, o TVS opera com 5.112 agentes autônomos divididos em 25 setores estratégicos — desde aeroespacial e defesa até saúde, finanças e educação.

O sistema foi construído em TypeScript/Node.js e funciona com 290+ provedores de IA (incluindo modelos locais via Ollama para operação offline completa). Gera tokens ERC-20, compila aplicações móveis Android/iOS, cria executáveis desktop, e se auto-evolui a cada 30 minutos com incrementos de +500% de inteligência.

DADOS PRINCIPAIS

- 25 Setores Estratégicos de atuação
- 290+ Provedores de IA via OmniRoute Bridge
- Auto-Evolução: +500% de inteligência a cada 30 minutos
- Tokens: \$VSR (300M supply) e \$TRIN (utilidade)
- Deploy multiplataforma: Web, Mobile (Android/iOS), Desktop (Windows/Mac/Linux)

AIProviderBridge	Fábrica de provedores: Ollama, OpenAI, Claude, Gemini, Grok, OmniRoute
ProviderFactory	Memória de curto prazo (100 itens/sessão) e longo prazo (persistente em JSON)
MemoryEngine	Roteia requisições para o modelo de IA ótimo entre 8+ provedores
ModelRouter	Gerencia ciclo de vida de todos os agentes (registro, execução, status)
AgentManager	Coordena tarefas multi agente com decomposição automática de subtarefas
Orchestrator	
ViseronCore	Motor principal que orquestra todos os componentes do sistema

2. Arquitetura do Sistema

3. Comandos de Inicialização

Comandos para iniciar, testar e verificar o sistema.

Comando	Descrição	Ação
npm start	Inicia o sistema TVS (produção)	node dist/src/index.js
npm run dev	Modo desenvolvimento com hot-reload	nodemon --exec tsx src/index.ts
npm run super:start	Inicia com superinteligência ativada	tsx src/index.ts
npm run launch	Executa o script de lançamento do mercado	tsx src/launch/market.ts
npm run setup	Instala dependências do root e mobile	npm install && cd mobile && npm install
npm run init	Executa script de inicialização	scripts/init-system.ps1
npm run init:full	Build + Backup + Start (ciclo completo)	scripts/init-system.ps1 -Full

Script init-system.ps1

Uso: ./scripts/init-system.ps1
[parâmetros]
-Build : Compila TypeScript
-Start : Inicia o sistema
-Backup : Executa backup diário
-Full : Build + Backup + Start
(ciclo completo)

4. Comandos de Build & Compilação

Comando	Descrição	Ação
npm run build	Compila TypeScript para dist/	tsc + copia assets
npm run lint	Verifica erros de TypeScript	tsc --noEmit
npm test	Executa testes core	tsx tests/core.test.ts
npm run test:hyper	Executa testes hyperlearning	tsx tests/hyperbrain.test.ts

Build do AIOX Core CLI

cd packages/aiox-core && npm run build

5. Comandos de Deploy & Publicação

Comandos para publicar o sistema em todas as plataformas.

Comando	Descrição	Ação
npm run deploy	Deploy script manual	scripts/deploy-all.ps1
npm run deploy:full	Build + Backup + GitHub + Vercel	scripts/deploy-all.ps1 -Full
npm run deploy:github	Push para GitHub apenas	scripts/deploy-all.ps1 -GitHub
npm run deploy:vercel	Deploy landing page para Vercel	scripts/deploy-all.ps1 -Vercel

Plataformas de Deploy

- GitHub : <https://github.com/ViseronSystem/trinnity-viseron-system>
- Vercel : <https://trinnityviseron.com> (Landing Page)
- Railway : Backend TVS Core (configurado em railway.json)
- Docker : docker-compose up (TVS + Ollama + Qdrant + n8n)

6. Comandos Mobile (Android & iOS)

Comando	Descrição	Ação
npm run build:android	Build APK para Android	build-all.ps1 -Target android
npm run build:ios	Build IPA para iOS (macOS)	build-all.ps1 -Target ios
npm run build:all	Build Android + iOS	build-all.ps1 -Target all
npm run build:eas-android	Build via EAS (Expo)	build-all.ps1 -Target eas-android
npm run build:eas-ios	Build iOS via EAS (Expo)	build-all.ps1 -Target eas-ios
npm run mobile:start	Iniciar Expo dev server	cd mobile && npx expo start
npm run mobile:android	Run Android direct	cd mobile && npx expo run:android
npm run mobile:ios	Run iOS direct	cd mobile && npx expo run:ios

Estrutura do App Mobile

Framework: Expo SDK 52 + React Native 0.76.9
Telas: Dashboard, Agents, Terminal (navegação por abas)
Conexão: Socket.IO + REST API para o backend TVS
Build: EAS Build (expo.dev) para APK e IPA

7. Comandos Desktop (Electron & Standalone)

Comando	Descrição	Detalhes
npm run build:exe	Build executável standalone (win)	scripts/build-standalone.mjs
npm run build:exe:win	Build para Windows	--platform win
npm run build:exe:mac	Build para macOS	--platform mac
npm run build:exe:linux	Build para Linux	--platform linux
npm run build:exe:all	Build para todas plataformas	--platform all
npm run build:electron	Build Electron (win)	--platform win --electron
npm run build:electron:all	Build Electron todas	--platform all --electron
npm run electron:setup	Instalar deps do Electron	cd electron && npm install
npm run electron:start	Iniciar Electron	cd electron && npx electron .
npm run electron:build:win	Build Electron Windows	NSIS installer + portable
npm run electron:build:mac	Build Electron macOS	DMG
npm run electron:build:linux	Build Electron Linux	ApplImage + deb

Saída dos Builds

- Standalone: .build/tvs-standalone/tvs-viseron-win.exe
- Portable: .build/TVS_Viseron_Portable.zip
- Electron: electron/dist-electron/ (configurável)

8. Comandos de Integrações

[illegible]

9. Comandos de Backup & Manutenção

Comando	Descrição	Script
npm run backup	Executa backup manual agora	scripts/backup-system.ps1
npm run backup:schedule	Agenda backup diário (03:00)	scripts/schedule-backup.ps1

Sistema de Backup Diário

O sistema de backup automático (scripts/backup-system.ps1) cria backups completos:

- Backup completo: config/, data/, database/, src/, scripts/, agents/, packages/, mobile/, electron/, docs/
- Formato: ZIP com timestamp (YYYY-MM-DD_HHmmss.zip)
- Retenção: 30 dias (backups antigos são removidos automaticamente)
- Agendamento: Windows Task Scheduler às 03:00 (ou manual via npm run backup)
- Inclui todos os PDFs e documentação

Localização dos Backups:
backups/YYYY-MM-DD_HHmmss.zip

10. Comandos do AIOX Core CLI

AIOX Core é a CLI oficial do ecossistema Trinnity Viseron.

Comando	Descrição	Detalhes
aiox-core init	Inicializa um novo projeto TVS	Cria estrutura base
aiox-core install	Instala dependências do projeto	npm install automático
aiox-core status	Mostra status do sistema TVS	Saúde e métricas
aiox-core --help	Ajuda completa da CLI	Todos os comandos

Instalação do AIOX Core

```
cd packages/aiox-core
npm install
npm link (ou npm install -g .)
```


11. API REST - Endpoints Completos

A API REST do TVS permite controlar todo o sistema remotamente. Dashboard em <http://localhost:3000>, Report Server em <http://localhost:3001>.

Dashboard API (Porta 3000)

Endpoint	Descrição	Resposta
GET /api/health	Health check do sistema	Status do servidor
GET /api/stats	Estatísticas completas	Agentes, memória, evolução
GET /api/agents	Lista todos os agentes	Array de agentes
GET /api/status	Status com squads	Squads e líderes
GET /api/battalion	Relatório do batalhão	114 agentes do batalhão
GET /api/battalion/:id	Agente específico	Detalhes do agente
GET /api/directives	Estatísticas de diretivas	Ativas e completadas
POST /api/directive	Emitir nova diretiva	Criar missão
POST /api/synthesize	Síntese multi-provedor	Ensemble de IAs

Report Server API (Porta 3001)

Endpoint	Descrição	Resposta
GET /	Informação do servidor	Endpoints disponíveis
GET /stats	Estatísticas do sistema	Agentes ativos/total
GET /agents	Lista compacta de agentes	ID, nome, role, status
GET /agents/:id	Detalhes de um agente	Capacidades completas
GET /report	Relatório JSON completo	Sistema + agentes + IA
GET /report/pdf	Download PDF do relatório	PDF formatado
GET /report/comprehensive-pdf	PDF completo do batalhão	PDF com todos os dados
GET /superintelligence	Status SuperIntelligence	Nível de inteligência
GET /supermind	Nível SuperMind	Domínios de conhecimento

ToolManager	Criação e execução de ferramentas externas	addShortTerm(), setLongTerm()
MemoryEngine	Memória STM (100 itens) + LTM (persistente)	modelRouter.route(criteria)
ModelRouter	Roteamento inteligente para o melhor modelo IA	agentManager.list(), register(), run()
AgentManager	Registro, ciclo de vida e execução de agentes	
O TVS possui 22 subsistemas core no ViseronCore, cada um com responsabilidades específicas:		

12. Módulos do Core TVS

13. Provedores de IA Suportados

O TVS suporta 8+ provedores de IA diretamente e 290+ via OmniRoute Bridge. Abaixo os provedores nativos:

Provedor	Modelos	Custo/1K	Contexto	Diferencial	Requisito
Ollama (Local)	Llama 3, Qwen 2, Mistral	0	8K-32K	Offline, zero custo	Padrão
OpenAI	GPT-4o, GPT-4o-mini, o1	\$0.002-0.015	128K-200K	Estado da arte	API Key
Anthropic	Claude Sonnet 4, Opus 4	\$0.003-0.015	200K	Segurança, contexto longo	API Key
Google	Gemini 2.5 Flash/Pro	\$0.00015-0.00125	1M+	Multimodal, mais rápido	API Key
xAI	Grok 3	\$0.002	131K	Dados em tempo real	API Key
DeepSeek	DeepSeek Chat	\$0.0005	128K	Open-weight, competitivo	API Key
Mistral	Mistral Large/Small	\$0.001-0.002	32K-128K	Eficiente, EUA	API Key
Cohere	Command A	\$0.0015	128K	RAG-otimizado	API Key
HuggingFace	Llama 3.3 70B, Mixtral	\$0.0004-0.0005	8K-65K	Open-source	API Key
Together AI	Llama 3.3 70B Turbo	\$0.0005	8K	Inferência rápida	API Key
Perplexity	Sonar Pro	\$0.001	128K	Pesquisa online	API Key
OmniRoute	290+ modelos agregados	Variado	Variado	Gateway universal	Config

Estratégias de Roteamento

- single: Usa um único provedor (default: Ollama)
- compare: Compara respostas de múltiplos modelos
- ensemble: Agrega respostas de todos os provedores disponíveis
- fallback

ack:
Se um
falha, p
róximo
da lista
é
usado
automa
ticamente

•
local-
first:
Tenta
Ollama
primeir
o,
depois
cloud

14. Tokens: \$VSR e \$TRIN

\$VSR — Viseron Crown (Token de Governança)

Supply: 300,000,000 VSR

Standard: TVS Standard v1.0.0

Função: Governança do sistema, voto em evoluções, diretivas

Distribuição:

- Trinnity Hurtado (Tesouro Corona): 90M VSR (30%)
- Pedro Costa (Tesouro Hierro): 75M VSR (25%)
- TVS Legion (Pool de Agentes): 90M VSR (30%)
- Reserva Estratégica: 45M VSR (15%)

\$TRIN — Trinnity (Token de Utilidade)

Supply: Dinâmico (cunhado/queimado por atividade)

Função: Gas para execução de agentes, créditos de computação, taxas

Rede: Ethereum (ERC-20)

Tokenomics:

- Distribuição: Team 10%, Marketing 15%, Liquidity 20%, Development 10%, Staking 25%, Community 20%
- Inflação: 2% anual
- Deflação: 1% queimado por transação

Criação automática de aplicações web, mobile (Android APK + iOS IPA), desktop (Windows/Mac/Linux executáveis) e Docker containers a partir de descrições em linguagem natural.
Ø=Un Geração Multi-Plataforma
15. Soluções que Oferecemos ao Mundo
Acesso a 200+ provedores de IA através de um único API, roteamento inteligente, failback automático e otimização de custos. Sem vendor lock-in.
Ø<B Gateway Universal de IA (OmniRoute)
5.112 agentes autônomos trabalhando em paralelo para resolver problemas complexos. Cada agente é especializado em um domínio específico, permitindo análise multidimensional de qualquer desafio.
Ø=V Superinteligência Multi-Agente
O Trinity Vison System não é apenas um software — é uma plataforma completa de Superinteligência que resolve problemas reais em escala global. Abaixo, as soluções que entregamos ao mundo:

O QUE VOCÊ RECEBE

16. Modelo de Assinatura — Por que Pagar?

- 5.112 Agentes de IA trabalhando 24/7 para você

' Acesso a 290+ modelos de IA (GPT-4o, Claude, Gemini, Grok, etc.)

' Geração ilimitada de aplicações (web, mobile, desktop)

' Criação de tokens e contratos inteligentes

O Trinity Vision System é um sistema operacional de superinteligência que substitui dezenas de ferramentas, serviços e equipes. Abaixo, explicamos por que o modelo de assinatura é necessário e justo.

' Sistema de chamadas com IA via Twilio

' Armazenamento de memória de longo prazo

' Atualizações contínuas e auto-evolução

' Suporte prioritário da equipe TVS

' Execução local (offline) ou cloud

POR QUE NÃO É GRATUITO?

Infraestrutura de IA

Cada requisição a modelos como GPT-4o, Claude Opus ou Gemini Pro tem custo real por token. O TVS otimiza esses custos, mas eles existem.

Manutenção Contínua

5.112 agentes, 22 módulos core, 6 bridges de integração — tudo requer atualização, testes e suporte contínuos.

Desenvolvimento Constante

Novas funcionalidades, mais provedores, melhor performance. O investimento em P&D é contínuo e significativo.

Infraestrutura de Servidores

Dashboard, API, Report Server, MCP Server, n8n, Qdrant vector DB — múltiplos serviços rodando 24/7.

Suporte e Documentação

Equipe dedicada para suporte técnico, documentação, tutoriais e resolução de problemas.

Economia de Escala

Por \$29-99/mês você substitui: equipe de 10+ desenvolvedores, 5+ assinaturas de IA, 3+ serviços de cloud.

DEVELOPER

17. Planos e Preços

Três planos simples para atender desde desenvolvedores individuais até grandes empresas.

19. Glossário de Comandos Rápidos

Resumo executivo dos comandos mais importantes para o dia a dia.

Categoria		
	Comando	Descrição
Ø=Þ€ INICIAR	npm run dev	Modo desenvolvimento com hot-reload
Ø=Þ€ INICIAR	npm start	Modo produção
Ø=Þ€ INICIAR	npm run init:full	Build + Backup + Start completo
Ø=ÞàÞ BUILD	npm run build	Compilar TypeScript
Ø=ÞàÞ BUILD	npm run lint	Verificar erros TS
Ø=Üñ MOBILE	npm run build:android	Gerar APK Android
Ø=Üñ MOBILE	npm run build:ios	Gerar IPA iOS
Ø=Ü» DESKTOP	npm run build:exe	Gerar .exe Windows
Ø=Ü» DESKTOP	npm run build:electron	Build Electron
Ø=Üä DEPLOY	npm run deploy:full	GitHub + Vercel completo
Ø=Üä DEPLOY	npm run deploy:github	Push GitHub
Ø=Üä DEPLOY	npm run deploy:vercel	Deploy Vercel
Ø=Ü¼ BACKUP	npm run backup	Backup manual imediato
Ø=Ü¼ BACKUP	npm run backup:schedule	Agendar backup 03:00
https://github.com/ViseronSystem/trinnity-viseron-system		
Ø=Y INTEGRAÇÕES	npm run omniroute:start	Gateway 290+ IAs
Ø=Y INTEGRAÇÕES	npm run twilio:call	Call System Twilio
Ø=Y INTEGRAÇÕES	npm run jarvis:start	OpenJarvis Bridge
"Construindo o futuro da inteligência artificial — uma mente de cada vez."		
Ø=Y INTEGRAÇÕES	npm run asno:start	ASNO JARVIS
Ø>Yê TESTES	npm test	Testes core do sistema
TRINNITY VISERON SYSTEM v5.0		
Ø>Yê TESTES	npm run test:hyper	Testes hyperlearning
Ø=ÜÊ RELATÓRIOS	http://localhost:3000	Dashboard web
Ø=ÜÊ RELATÓRIOS	http://localhost:3001/report/pdf	PDF do sistema



